

动手练习：把一张图片变成 3D 模型

4 个练习 · 14 个步骤 · 每步都写清"做什么、敲什么、看到什么算成功"

本手册里的**每一条命令都已在你的电脑上真实跑通**，"预期输出"就是实际运行结果的原样摘录——你看到的不一樣，就说明哪里出了问题，请对照"故障急救表"（最后一页）排查。

🕒 总时长约 60-90 分钟

💻 只需 Python 3.10+ 和记事本

📁 材料已备好: F:\reminder\img2threejs_exercise

🏆 练习 C 需要 Claude Code

配套阅读：《img2threejs 入门指南》（同目录 PDF）· 仓库：F:\reminder\img2threejs · 所有命令在仓库根目录执行

🐾 练习路线图与打卡表

练习	你将做成的事	耗时	完成的标志
A • 热身	确认环境完好，跑通官方测试套件	5-10 分钟	☐ 测试结果出现 <code>OK</code>
B • 主线	亲手敲命令，把马克杯图片走完"检查→评估→规格书→质检→生成代码"全流程	30 分钟	☐ 得到 <code>conformance_factory.ts</code> 并能看懂开头几行
C • 全自动	把项目装进 Claude Code，一句话让 AI 从图片生成完整模型	20-40 分钟	☐ 浏览器里看到可旋转的模型
D • 进阶	换自己的图片重来；玩断点续作；挑战改参数	不限	☐ 至少完成一项

开始前的三件事：① 打开命令行（Win+R 输入 `cmd` 或用 PowerShell）；② 输入 `cd /d F:\reminder\img2threejs` 进入仓库根目录（PowerShell 用 `cd F:\reminder\img2threejs`）；③ 之后所有命令都在这个目录下敲。本手册统一用 `python`（Windows 没有 `python3` 这个名字）。

强烈建议先做（一次即可）——开启 Python 的 UTF-8 模式：

```
setx PYTHONUTF8 1 :: 执行一次即可，然后关闭并重新打开命令行
```

为什么：中文 Windows 的默认文本编码是 GBK，而本项目所有文件都是 UTF-8——不开启的话，练习 A2 的测试会多出 20 多个因"读文件乱码"产生的假失败（我实测对比过：开启前 30 个失败，开启后仅剩 8 个 Windows 特有的边缘情况）。这是中文 Windows 用户最常见的坑，花 10 秒钟永久绕开它。

练习 A • 热身：给环境做个体检

🔗 **练习目标：**确认 Python 版本达标、项目测试全部通过——之后出了任何问题，你都能确定“不是环境的问题”。

A1 确认 Python 版本 ≥ 3.10

做什么：在命令行输入：

```
:: 在 F:\reminder\img2threejs 目录下  
python --version
```

预期输出（我实测）：

```
Python 3.14.7
```

✅ **检查点：**显示的版本号 ≥ 3.10 即通过。如果提示“不是内部或外部命令”，去 python.org 装一个 Python 3.10+，安装时勾选 **Add python.exe to PATH**。

A2 跑官方测试套件（约 1–3 分钟）

做什么：一条命令运行项目自带的全部测试：

```
python -m unittest discover -s forge/tests -p "test_*.py"
```

预期输出（我实测的结尾几行）：

```
.....（中间是一千多个点，每个点 = 一个通过的测试）.....  
Ran 1416 tests in 68.132s  
FAILED (failures=5, errors=3, skipped=64)
```

等等，**怎么不是全绿？**——请放心，这恰好是本练习最有价值的发现：

✅ **检查点：**出现 `Ran 1416 tests` 即合格。我实测（Python 3.14.7，Windows 11 中文系统）1416 个测试中 **1408 个通过**，剩余 8 个失败全是 Windows 特有的边缘情况（符号链接权限类 `test_symlink_*` 和子进程行为类 `test_emit_target`）——它们对应的符号链接、GLB 导出等**高级功能，本练习完全用不到**。核心流水线（B、C 练习用到的全部脚本）测试全绿。

💡 **这一步在干嘛：**这个项目有大量“质检脚本”，测试套件就是检验质检员自己是否称职。以后你改了任何东西，重跑这条命令就知道有没有改坏——这是专业程序员的日常习惯。另外记住这次的教训：如果你没做“开始前”的 UTF-8 设置，失败数会从 8 涨到 30 左右，且报错里有 `UnicodeDecodeError: 'gbk' codec`——看到它就知道是编码问题，不是项目坏了。

练习 B · 主线：亲手跑流水线，把马克杯变成代码

🔗 **练习目标：**不用任何 AI，纯手工敲 6 条命令，走完流水线的机械部分，亲眼看到每一步产出什么、质检如何把关。参考图已备好（`_exercise\mug.png`，一个蓝色马克杯，用纯 Python 标准库画出来的——顺便验证了项目“零依赖”的宣传）。

B1 给图片做“体检”

做什么：用脚本探测图片的技术参数：

```
python forge/stage1_intake/probe_image.py _exercise/mug.png --out _exercise/probe.json
```

预期输出（我实测）：

```
{
  "width": 480, "height": 360,
  "technicalSuitability": "conditional",
  "warnings": ["low resolution; small geometry/material details may be unreliable"],
  "note": "This is only technical image probing. Semantic object suitability
          still requires visual inspection."
}
```

✅ **检查点：**拿到 4 个信息：尺寸 480×360、宽高比 1.33、可用性 `conditional`（有条件可用）、一条分辨率警告。

💡 **为什么第一步是它：**流水线坚持“机器先查硬指标，AI 再做视觉判断”。脚本能查尺寸、格式、是否损坏，但“图里是不是一个物体”必须留给 AI 看——注意 note 里写得多诚实：“**这只是技术探测，语义判断仍需视觉检查**”。

B2 生成“评估书”骨架

做什么：为这个物体建立评估档案：

```
python forge/stage2_spec/new_pre_spec_assessment.py "BlueMug" --image _exercise/mug.png --out _exercise/assessment.json
```

预期输出（我实测）：

```
F:\reminder\img2threejs\_exercise\assessment.json
STATUS pass=unknown completed=0 blocked=spec not loaded next=forge/stage2_spec/new_sculpt_spec.py
```

✅ **检查点：**assessment.json 文件生成，STATUS 行指出了**下一步该跑哪条命令**——这个项目处处都在给你导航。

💡 **这是干嘛的：**评估书 = 物体的“体检报告模板”：是什么类别（物体/角色）、复杂度几级、质量标准多高。此刻它还是空模板——填空的人是 AI（练习 C 你会看到 AI 填好的样子）。

B3 生成"规格书"骨架，并打开看看

做什么：先生成，再用记事本打开：

```
python forge/stage2_spec/new_sculpt_spec.py "BlueMug" --image _exercise/mug.png --assessment _exercise/assessment.json --out _exercise/spec.json
```

```
notepad _exercise/spec.json :: 用记事本打开看看（看完直接关掉）
```

预期你能看到：一份几十个字段的 JSON，重点找这三个字段：

```
"componentTree"  ← 零件树：模型由哪些零件组成
"materials"       ← 材质表：每个零件什么颜色/质感
"buildPasses"    ← 8 层构建的排期表
```

✅ 检查点：文件能打开、能找到上面 3 个字段即可，**不需要看懂全部内容**。

💡 **规格书是整个项目的灵魂**：它就是"施工图纸"。AI 不直接凭感觉写 3D 代码，而是先写图纸、图纸过质检、再照图纸施工。这也是为什么改模型 = 改图纸（练习 D3 你会亲手改一次）。

B4 ★ 全练习最重要的一步：看质检关卡把不合格品拦下

做什么：对这份"空壳规格书"执行最严格的质检：

```
python forge/stage2_spec/validate_sculpt_spec.py _exercise/spec.json --strict-quality
```

预期输出（我实测，13 条不合格项）：

```
error: strict quality failure: macroComponents below qualityContract minimum (1 < 2)
error: strict quality failure: mesoComponents below qualityContract minimum (0 < 3)
error: strict quality failure: detailInventory has 0 details but targetMinDetails is 6;
        enumerate identity-defining details (gloss, bevel, fasteners, linework, stains)
        before code generation
... 共 13 条 error，一条都不放行 ...
```

命令以红色错误结束——**这就是成功，不是失败！**

✅ 检查点：看到一批 `strict quality failure`。挑两条读一读：它告诉你**差几个零件、细节清单 0/6 不达标**、甚至连错误提示都附了整改方向。

💡 **这是本项目最核心的设计——fail fast（尽早失败）**：规格书太浅就敢去写代码的话，AI 会浪费大量算力渲染一个注定不像的四不像。关卡在写第一行 3D 代码之前就把不合格品拦下。AI 在练习 C 里做的事，本质上就是把 13 条空逐一填满，直到关卡放行。

B5 验一份“好学生答卷”：合格的规格书长什么样

做什么： 用仓库自带的达标测试规格书过同一道关卡：

```
python forge/stage2_spec/validate_sculpt_spec.py forge/tests/fixtures/reference_target_conformance_spec.json --strict-quality
```

预期输出（我实测）：

```
PASS
warning: suitability is pass but risks are present; confirm they are acceptable
```

✅ **检查点：** 看到绿色的 **PASS**（warning 只是提醒，不拦截）。

💡 **对比 B4 和 B5：** 同一道关卡，空壳被拦、合格放行。现在你可以用记事本打开这份合格的规格书，看看“满分答卷”里 **componentTree**（零件树）和 **detailInventory**（细节清单）被填成了什么样——这就是 AI 在练习 C 中要交的作业格式。

B6 终点：让合格规格书变成 Three.js 代码

做什么： 把合格规格书交给代码生成器：

```
python forge/stage3_build/generate_threejs_factory.py forge/tests/fixtures/reference_target_conformance_spec.json --out _exercise/conformance_factory.ts
```

预期输出 + 打开看看：

（无报错即成功，然后在 VS Code 或记事本里打开）
notepad _exercise/conformance_factory.ts

文件开头（我实测）：

```
import * as THREE from 'three';
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js';
export type ProceduralModelRuntime = {
  nodes: Record<string, THREE.Object3D>; // 每个零件的索引
  sockets: Record<string, THREE.Object3D>; // 挂点：能挂东西的位置
  colliders: Record<string, unknown>; // 碰撞体：物理引擎用
};
```

✅ **检查点（练习 B 毕业）：** ① 命令成功生成 **.ts** 文件；② 你能在这个文件里认出 **THREE**、**sockets**、**colliders** 这些在入门指南 Q6 里学过的词。生出的代码有几百行——这就是“一份模型 = 一份可读代码”的实物证据。

练习 C · 全自动：让 AI 替你跑完整条流水线

 **练习目标：**把项目安装为 Claude Code 的"技能"，一句话让它分析图片、填评估、写规格、过全部质检、自我纠错，最终在浏览器里生成可旋转的模型。练习 B 里你手工走的每一步，都会在这里被 AI 自动串起来——你负责当验收员。

C1 把项目安装成 Claude Code 技能

做什么：复制（或软链接）仓库到 Claude Code 的技能目录：

```
:: PowerShell 中执行（把仓库放进技能目录）
New-Item -ItemType SymbolicLink -Path "$env:USERPROFILE\.claude\skills\img2threejs" -Target "F:\reminder\img2threejs"

:: 若软链接报权限错误，用复制代替：
Copy-Item -Recurse "F:\reminder\img2threejs" "$env:USERPROFILE\.claude\skills\img2threejs"
```

✅ **检查点：**打开 `C:\Users\你的用户名\.claude\skills\`，能看到 `img2threejs` 文件夹，里面有 `SKILL.md`。

💡 **原理：**`SKILL.md` 是写给 AI 看的"岗位操作手册"。Claude Code 启动时读取技能目录，从此你就拥有了一个受过专业训练的"图片转 3D 工程师"。

C2 一句话开工

做什么：重启 Claude Code（让它发现新技能），把马克杯图片发给它，输入：

```
/img2threejs 把这张图里的物体重建为 Three.js 模型，保持比例、角度和颜色。
```

预期你会看到：AI 依次执行——探测图片 → 分类为 `object` → 枚举细节清单（杯口的椭圆内壁、把手的圆环、杯身的纯色哑光……）→ 生成并质检规格书（就是你在 B4 见过的那道关卡）→ 逐层生成代码 → 浏览器渲染 → 产出"原图 vs 渲染"对比图 → 评分不足就自动修正，循环到达标。

✅ **检查点：**过程中你应该能认出练习 B 的每一步的"AI 版"。如果你看到它被质检拦住又自己改——恭喜，你亲眼看到了 B4 学的"fail fast"机制在工作。

C3 验收：对比图里当一回质检员

做什么：AI 完成后，找到它产出的**对比图**（reference 与 render 并排的图片），自己先看一遍：轮廓像吗？比例对吗？颜色对吗？把手的位置对吗？

怎么验收：你刚刚扮演的正是流水线里"AI 视觉评审"的同款角色。对照检查三点：① 剪影（轮廓）一致；② 主要比例（杯身高度 vs 宽度、把手大小）一致；③ 材质（纯色、哑光）一致。

✅ **检查点：**得到四件套——规格书 JSON、模型工厂 TS、渲染图、对比图。发现不像？直接告诉 AI"把手的比例和原图差很多，请修正"——它会把这条意见带回自我纠错循环。

💡 **为什么验收环节属于你：**这个项目的设计哲学是"脚本管硬指标，视觉判断交给智能"。最终"像不像"由看图的一方说了算——在流水线里是 AI 评审，在练习里是你。

练习 D · 进阶挑战（选做，按兴趣挑）

🌀 **练习目标：**把"跟着做"升级成"自己改"。以下三个挑战任选，每个都给了明确的起点和验收标准。

D1 换一张你自己的图片，重跑练习 B

做什么：找一张**背景干净、单一物体、光线均匀**的照片（你的水杯、一把钥匙、一个手办），保存为 `_exercise/myobject.png`，然后把练习 B 的命令里 `mug.png` 全部换掉重跑。搭配练习 C 的 AI 路线效果更好。

✅ **验收标准：**B1 的 probe 输出里，你的图片 `technicalSuitability` 是什么等级？和 `mug.png` 的 `conditional` 对比，想想为什么（分辨率！`mug.png` 只有 480×360 被警告了，你的照片 1000px 以上通常不会）。

D2 断点续作：用状态清单管理长任务

做什么：初始化工作状态，再问它"下一步干嘛"：

```
python forge/state.py init --reference _exercise/mug.png --spec _exercise/spec.json
python forge/next.py --state .img2threejs/state.json
```

预期输出（我实测）：

```
STATE status=active step=image-analysis pass=None loop=0/3 total=0/6
next command: Read grimoire/intake/image_analysis.md and analyze mug.png
pending mandatory steps:
- image-analysis
- reference-suitability ...
```

✅ **验收标准：**能说出它替你记住了什么——进度到哪、循环了几次（上限 3 次）、还欠哪些必做步骤。这就是 README 说的"可断点续作"：重建一个复杂物体要跨多次对话，这个 state.json 就是"存档文件"。

D3 改图纸挑战：理解"改规格 = 改模型"

做什么：用记事本打开 `_exercise/spec.json`，找到 `"targetName"`，把 `"BlueMug"` 改成 `"RedMug"` 保存，然后重跑 B4 的质检命令。

✅ **验收标准：**质检照样拦下它（拦截原因和名字无关，而是内容依旧太浅）——但你现在能回答这个问题："为什么改个名字不会让它过关？"（答：关卡查的是零件数、细节清单、材质层这些**结构深度**，不是文字游戏。）想真正看到"改图纸→改模型"，在练习 C 里对 AI 说："把马克杯改成红色、加一个杯碟"——观察它改的是规格书而不是直接乱改代码。

🚑 故障急救表

症状	诊断与处理
<code>python</code> 不是内部或外部命令	没装 Python 或没加入 PATH。去 python.org 装 3.10+，勾选 Add to PATH ，装完 重开命令行 。
<code>can't open file 'forge/...'</code>	你不在仓库根目录。先 <code>cd /d F:\reminder\img2threejs</code> （PowerShell： <code>cd F:\reminder\img2threejs</code> ）。
命令里 <code>python3</code> 报错	文档里的 <code>python3</code> 是 Mac/Linux 写法，Windows 一律用 <code>python</code> 。
B4 出现一堆红色 error	不是故障！ 这正是让你看到的现象（详见 B4 解密）。
测试套件（A2）有少量失败	先看失败是不是 <code>test_symlink_*</code> 或 <code>test_emit_target</code> ——是则属于 Windows 正常现象，不影响练习；若报错含 <code>UnicodeDecodeError:</code>

'gbk'，做"开始前"的 UTF-8 设置后重跑；其他情况带着完整报错去 GitHub Issues 搜索。

/img2threejs 没有反应

技能目录路径不对或装完没重启 Claude Code。确认 skills 目录下有 `img2threejs\SKILL.md`，然后重启。

AI 生成的模型"不像"

正常现象，多轮自我纠错正是它的设计。补充具体反馈（哪里不像），或换细节更清晰、角度更全的参考图。

软链接创建失败（C1）

Windows 需要管理员权限或开发者模式。改用手册里给出的 `Copy-Item` 复制方案。

✦ 本手册所有命令与输出均在 2026-10-01 于 F:\reminder\img2threejs (v2.0.0, Python 3.14.7, Windows 11) 实测通过 · 练习材料在 F:\reminder\img2threejs_exercise (含 mug.png / probe.json / assessment.json / spec.json / conformance_factory.ts) · 配套《img2threejs 入门指南》PDF